

7. Producer-Consumer (Bounded Buffer) Problem

Syllabus Coverage: Gateway Classes Slides 65–75 | Circular Buffer, Assembly Race Condition, 3 Semaphores Coordination, Deadlock Pitfall.

7.1 Buffer Architecture & Pointers

Circular array of size `N`. Producer places items at `in`, Consumer extracts items from `out`.

Producer-Consumer (Bounded Buffer) Problem

Circular Ring Buffer & Semaphore Coordination: empty, full, mutex

Shared Circular Buffer (Size N = 8)

X0
[0]

X1
[1]

X2
[2]

—
[3]

—
[4]

—
[5]

—
[6]

—
[7]

Pointers: `in = 3` (Next free slot) | `out = 0` (Next item to consume)

Variables: `count = 3` (Total items currently resident)

Hazard: Simultaneous `count++` & `count--` causes Race Condition!

Semaphores: `empty = 5`, `full = 3`, `mutex = 1`

Correct Semaphore Solutions

PRODUCER PROCESS:

```
wait(empty); // Decrement empty slots
wait(mutex); // Acquire buffer lock
buffer[in] = item; in = (in + 1) % N;
signal(mutex); // Release buffer lock
signal(full); // Increment full items
```

CONSUMER PROCESS:

```
wait(full); // Decrement full items
wait(mutex); // Acquire buffer lock
item = buffer[out]; out = (out + 1) % N;
signal(mutex); // Release buffer lock
signal(empty); // Increment empty slots
```

⚠ THE FAMOUS DEADLOCK TRAP (Exam & Interview Favorite)

Wrong Order: wait(mutex) before wait(empty)

Agar buffer full hai (`empty = 0`):

1. Producer wait(mutex) karke lock le leta hai (`mutex=0`).
2. Producer wait(empty) par BLOCK ho jata hai!
3. Consumer wait(mutex) par block hota hai $\rightarrow$ DEADLOCK!

Golden Rule: Signaling vs Locking Order

Resource availability check (`empty/full`) HAMESHA mutual exclusion lock (`mutex`) se PEHLE aana chahiye!

CORRECT: `wait(empty)` $\rightarrow$ `wait(mutex)`

7.2 Synchronization Semaphores

- `semaphore mutex = 1;` — Ensures mutual exclusion while touching buffer pointers.
- `semaphore empty = N;` — Counts available free slots.
- `semaphore full = 0;` — Counts items available for consumption.

7.3 The Critical Deadlock Trap

Reversing the wait calls to `wait(mutex); wait(empty);` results in an immediate Deadlock when the buffer is full, because the Producer holds the lock while sleeping, starving the Consumer from freeing any space.